

Análisis Exploratorio y Preparación de Datos

para el Fine-Tuning de mBERT y XLM-R

Corpus Salvadoreño de Toxicidad y Discurso de Odio

Universidad de El Salvador
Facultad de Ingeniería y Arquitectura
Escuela de Sistemas Informáticos

Julio Enrique De La Quadra Mata
Cristian Antonio Escalante Hernández
Willian Alexander Chávez Servellón

Etapa II · 30 de julio de 2026

Resumen ejecutivo

Este informe deja el corpus listo para el *fine-tuning* y responde con datos las preguntas que condicionan esa fase. El dataset final tiene **3,063 publicaciones** en cuatro clases, con las cuatro por encima del mínimo del dimensionamiento estadístico. Tres resultados condicionan el diseño experimental: **(1)** X y Facebook son distribuciones estadísticamente distintas, lo que obliga a estratificar y a reportar métricas desagregadas; **(2)** el desbalance de clases es moderado (ratio 1.93 a 1) y las cuatro superan el mínimo del dimensionamiento, aunque dos quedan por debajo del escenario de alta precisión; **(3)** la jerga salvadoreña está asociada a toxicidad con un *lift* de 1.81x, un riesgo de sesgo que hay que medir explícitamente.

1. Alcance y criterio

El objetivo del proyecto es clasificar toxicidad y discurso de odio en publicaciones salvadoreñas de X y Facebook mediante *fine-tuning* de **mBERT** y **XLM-R**, con meta de **F1-macro superior al 80%**. Este documento cubre la fase de *Preparación de los Datos* de CRISP-DM y organiza cada hallazgo alrededor de una pregunta con consecuencia práctica sobre el entrenamiento.

Pregunta	Sección
¿Cuántos datos hay por clase y alcanzan para F1-macro > 80%?	3
X y Facebook, ¿son distribuciones distintas? ¿generalizará el modelo entre ellas?	4
¿Qué tan dialectal es el corpus frente a los datasets públicos de España y México?	5
¿Qué <code>max_length</code> usar? ¿Cuánto pierde cada tokenizador?	7
¿Cómo se armonizan los datasets públicos para la primera etapa?	8

1.1 Criterio de normalización: preservación máxima

No se aplica el preprocesamiento clásico de PLN —minúsculas, quitar tildes, corregir ortografía, eliminar puntuación y *stopwords*, lematizar—. Esa receta pertenece a modelos de bolsa de palabras. Con Transformers preentrenados el planteamiento se invierte, por tres razones:

- **Desalineación con el preentrenamiento.** mBERT y XLM-R construyeron sus representaciones sobre español real, con tildes, mayúsculas y puntuación. El tokenizador *subword* ya modela esas formas.
- **La deformación ortográfica es señal.** Escribir pvto o 4güevo suele ser evasión deliberada de filtros de moderación: es el fenómeno a detectar, no ruido.
- *Train/serve skew.* En producción la API recibirá texto crudo. Toda transformación aplicada al entrenamiento debe replicarse idénticamente en inferencia.

Decisión de arquitectura

Por esa última razón la normalización vive en `src/features/normalization.py` y no en el notebook. Ese módulo es la **única fuente de verdad**: lo importan el EDA, el script de entrenamiento y el *endpoint* de FastAPI. Es la garantía estructural contra el *train/serve skew*.

Sí se normaliza lo que **no es lingüístico** y lo que identifica personas: Unicode NFC, caracteres invisibles, colapso de espacios, y placeholders para @menciones y URLs. Las menciones se **reemplazan, no se eliminan**: su presencia distingue un ataque dirigido de una afirmación general, y eso es señal predictiva legítima.

Variante exportada	Uso
<code>texto_modelo</code>	Entrenamiento y producción. Campo por defecto.
<code>texto_agresivo</code>	Grupo de control del estudio de ablación
<code>texto_original</code>	Trazabilidad y análisis cualitativo de errores

2. Integridad de los datos

Interesa sobre todo lo que puede **inflar las métricas de test** sin dejar rastro.

Chequeo	n	%
Filas totales	3,104	100
IDs duplicados	0	0.00
Textos duplicados tras normalizar	41	1.32
Duplicados con etiqueta contradictoria	0	0.00
Espacios sobrantes al inicio o final	0	0.00
Mojibake o desviaciones de Unicode NFC	0	0.00
Usuarios con más de una publicación	0	0.00

Implicación para el fine-tuning

Los duplicados se eliminan **antes** del *split*: si el mismo texto queda en *train* y en *test*, el modelo lo memoriza y el F1 sale inflado sin que nada lo delate.

Ningún usuario aparece más de una vez, así que **no hace falta** *group split* por autor. De haber concentración, un *split* aleatorio dejaría publicaciones del mismo autor a ambos lados y el modelo aprendería estilo en lugar de toxicidad.

El corpus está limpio en la capa superficial, pero los pasos de NFC y colapso de espacios se mantienen igualmente: la entrada de producción sí vendrá sucia y el pipeline debe ser idéntico en ambos extremos.

3. Distribución de clases y suficiencia muestral

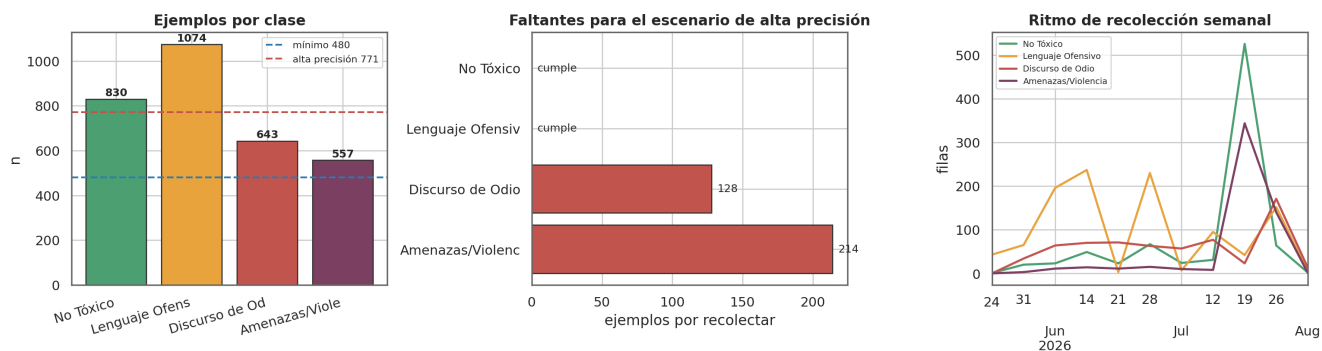


Figura 1. Ejemplos por clase frente a los umbrales del dimensionamiento estadístico, faltantes para el escenario de alta precisión y ritmo de recolección semanal.

Clase	n	%	Mínimo 480	Alta precisión 771
Lenguaje Ofensivo	1,074	34.6	cumple	cumple
No Tóxico	830	26.7	cumple	cumple
Discurso de Odio	643	20.7	cumple	faltan 128
Amenazas/Violencia	557	17.9	cumple	faltan 214

El ratio entre la clase mayoritaria y la minoritaria es de **1.93 a 1**: desbalance moderado. Las cuatro clases superan el mínimo de 480 ejemplos; dos alcanzan el escenario de alta precisión.

3.1 Piso de referencia sin modelo

Antes de entrenar nada conviene saber qué F1-macro produce un clasificador **trivial**, que no mira el texto y solo conoce la proporción de clases. No requiere entrenamiento: sale de aritmética sobre la distribución, así que pertenece legítimamente al análisis exploratorio.

Estrategia trivial	Cómo funciona	F1-macro
Clase mayoritaria	Predice siempre <i>Lenguaje Ofensivo</i>	0.129
Azar estratificado	Predice al azar respetando las proporciones reales	0.250
Azar uniforme	Predice al azar con 25% para cada clase	0.246

El mejor de los tres alcanza **0.250**. Frente al objetivo de 0.80 quedan **0.55 puntos** de F1-macro que solo pueden salir de aprender algo del texto. Es el **piso inferior** del proyecto: cualquier modelo que no lo supere holgadamente no está aprendiendo nada.

Por qué el mayoritario rinde peor que el azar

El azar estratificado da siempre **1/K** en F1-macro, sea cual sea la distribución —en este caso $1/4 = 0.25$ —, porque para cada clase la precisión y el *recall* coinciden con su proporción. La clase mayoritaria, en cambio, rinde *peor* (0.129) pese a acertar el 34.6% de las veces: acierta mucho en una clase y cero en las otras tres, y el promedio macro la castiga. Es exactamente el comportamiento que se quiere evitar en el modelo real, y la razón de usar F1-macro y no *accuracy*.

Implicación para el fine-tuning

Con **F1-macro** como métrica objetivo —promedia las clases sin ponderar por frecuencia— basta que una clase minoritaria falle para hundir el promedio. Estrategias en orden de preferencia:

1. `class_weight` en la función de pérdida. Sin coste de datos, primera opción. Se calcula **solo sobre train**: hacerlo antes del *split* filtra información del conjunto de prueba.
2. Recolección dirigida de las clases por debajo de 771. Es la solución real.
3. *Data augmentation* con las variantes ortográficas del léxico salvadoreño.
4. *Oversampling*: última opción. Con Transformers y textos cortos favorece la memorización.

Invertir en las clases minoritarias rinde más: cada punto de F1 en *Amenazas/Violencia* vale lo mismo que uno en *Lenguaje Ofensivo*, pero cuesta menos conseguirlo partiendo de 557 ejemplos.

4. Cross-platform: X frente a Facebook

Es un objetivo explícito del proyecto: «comparativa de rendimiento entre plataformas» y «que el modelo generalice entre plataformas». Si las dos son distribuciones distintas, hay **domain shift interno** y eso afecta al diseño experimental.

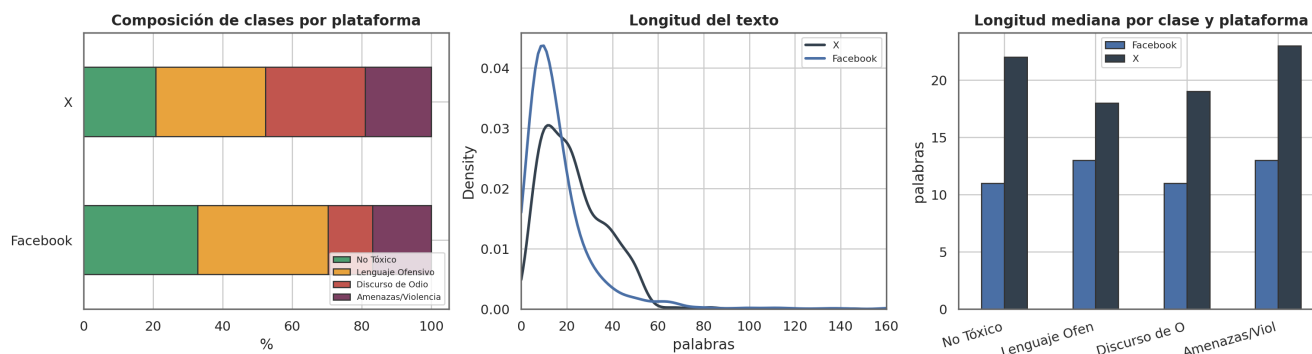


Figura 2. Composición de clases por plataforma, distribución de longitud del texto y longitud mediana por clase y plataforma.

Prueba	Resultado	Conclusión
Chi-cuadrado de independencia (plataforma × clase)	$\chi^2 = 145.1 \cdot gl = 3$ $p = 3.0 \times 10^{-31}$	La composición de clases no es independiente de la plataforma
V de Cramér	0.216	Asociación moderada
Mann-Whitney U (longitud del texto)	$p = 1.1 \times 10^{-62}$	Las distribuciones de longitud difieren

El dato contradice una hipótesis del perfil

El perfil del proyecto asume que «X se caracteriza por contenido corto y Facebook fomenta comentarios más extensos». **Los datos del corpus muestran lo contrario:** la mediana en X es de **20 palabras** y en Facebook de **12**.

No invalida el proyecto, pero sí es un supuesto del planteamiento que la evidencia no respalda y que conviene corregir en la tesina antes de que lo señale el jurado. La explicación probable es el sesgo de recolección: los comentarios capturados manualmente en Facebook tienden a ser respuestas cortas, no las publicaciones largas que el perfil tenía en mente.

Implicación para el fine-tuning

1. **Estratificar los splits por plataforma además de por clase.** Si *test* queda desbalanceado hacia una plataforma, el F1 medirá rendimiento en esa plataforma, no generalización. Ya está aplicado en la sección 10.
2. **Reportar F1-macro desagregado por plataforma,** no solo el global. Es exactamente el entregable del objetivo específico de comparativa X vs Facebook.
3. **Prueba de generalización cruzada:** entrenar en X y evaluar en Facebook, y viceversa. La caída de F1 en ese experimento es la medida del domain shift interno, y es un resultado publicable que responde directamente al objetivo del proyecto.
4. El `max_length` debe cubrir la cola larga, que según los datos está en X y no en Facebook.

5. Caracterización lingüística

Aquí se mide qué tan **salvadoreño** es el corpus. Importa porque la primera etapa del *transfer learning* usa datasets públicos orientados a España y México, y el perfil identifica ese sesgo dialectal como riesgo del proyecto.

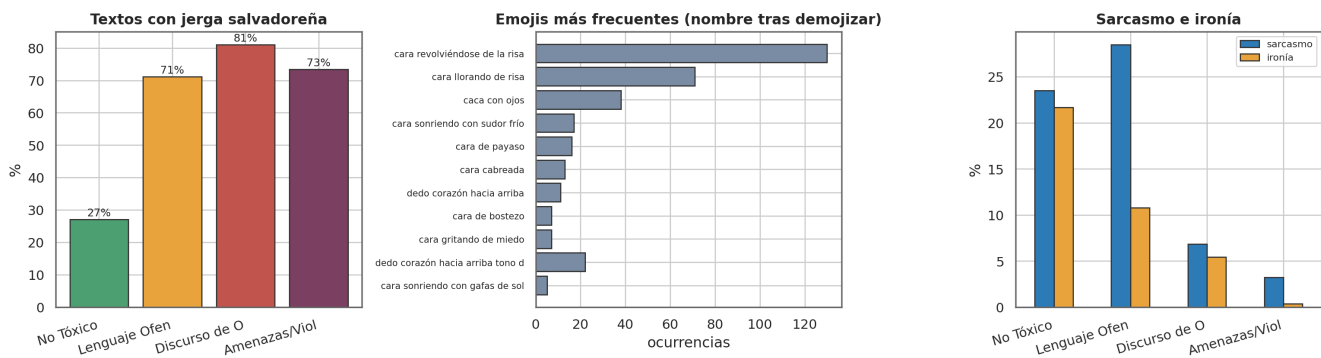


Figura 3. Presencia de jerga salvadoreña por clase, emojis más frecuentes con su nombre tras demojizar, y marcado de sarcasmo e ironía.

Indicador	Valor
Términos del lexicón usados como referencia	2,024
Textos con al menos un término local	61.8%
Media de términos locales por texto	1.09
Lift de toxicidad asociado a la jerga local	1.81x

El corpus es **efectivamente dialectal**: casi dos tercios de los textos contienen vocabulario salvadoreño registrado en el lexicón. Eso justifica la segunda etapa de *fine-tuning*, porque los datasets públicos no cubren este registro.

Riesgo de atajo dialectal

Pero el mismo dato encierra un riesgo. Un texto con jerga local tiene **1.81 veces** más probabilidad de ser tóxico que uno sin ella. Si esa asociación es fuerte, el modelo puede aprender el atajo «**modismo salvadoreño → tóxico**» en lugar de aprender toxicidad.

El resultado sería un clasificador que penaliza el habla coloquial salvadoreña —un sesgo directo contra los hablantes que el sistema pretende proteger— y no aparecería en el F1, porque el sesgo está también en el conjunto de prueba.

Implicación para el fine-tuning

Este es el análisis concreto que debe entrar en el **informe de fairness** que exige la rúbrica de la Etapa II (15 puntos):

1. Medir la tasa de falsos positivos sobre textos *No Tóxicos* que contienen jerga local frente a los que no la contienen. Una diferencia grande confirma el sesgo.
2. Comparar esa tasa antes y después de la segunda etapa de *fine-tuning*.
3. Cruzar los errores del modelo contra las columnas de sarcasmo e ironía, que el corpus ya tiene etiquetadas. Cubre el objetivo «*análisis de errores cualitativo — jerga salvadoreña*».

Sobre los **emojis**: se convierten a su descripción en español antes de tokenizar. El vocabulario de mBERT se construyó sobre Wikipedia, que casi no contiene emojis, y la mayoría caería en [UNK]. Un [UNK] colapsa todos los emojis en el mismo vector y borra la distinción entre burla, desprecio y agresión.

6. Verificación de la normalización

Comprobación de que el pipeline hace exactamente lo que declara:

CRUDO : Sos un [payaso] hermano, @juanperez mirá esto https://x.com/a/b
MODELO : Sos un :cara_de_payaso: hermano, @usuario mirá esto http://url
AGRESIVO : sos un hermano usuario mira esto url

Propiedad	Crudo	texto_modelo	texto_agresivo
Textos con tildes o ñ	1,825	1,825	0
Textos con mayúsculas	2,884	2,884	0
Textos con puntuación	2,538	2,538	0
Textos con @mención literal	147	0 (→ @usuario)	0
Textos con URL literal	54	0 (→ http://url)	0
Textos con emoji crudo	195	0 (→ texto en español)	0
Textos vacíos tras normalizar	0	0	0

Las tildes, mayúsculas y puntuación se conservan íntegramente en `texto_modelo`. La variante agresiva las destruye por diseño: existe solo como grupo de control.

7. Tokenización: mBERT frente a XLM-R

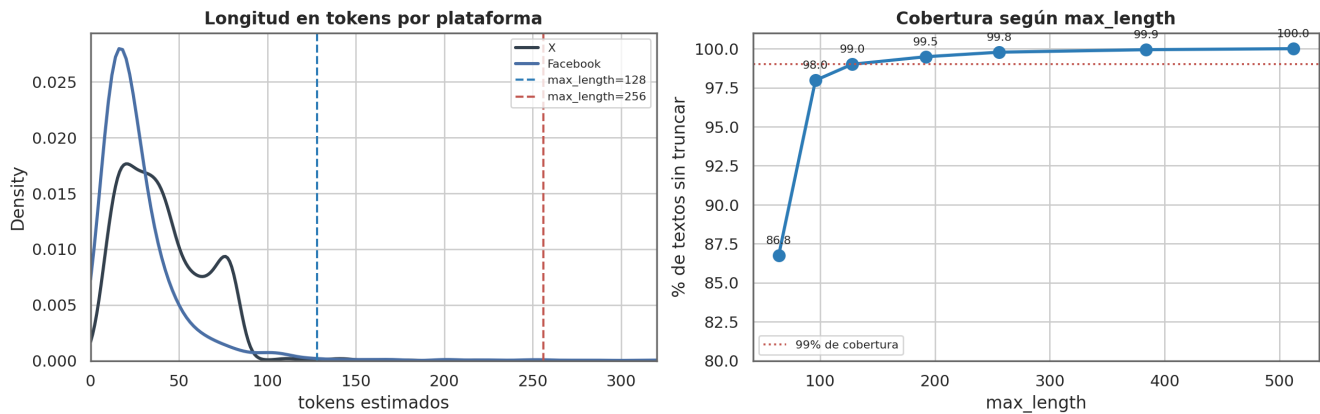


Figura 4. Longitud en tokens por plataforma frente a los cortes candidatos, y cobertura del corpus según el valor de `max_length`.

Decisión	Valor	Motivo
<code>max_length</code>	128	Cubre por encima del percentil 99. Subir a 512 multiplica ~16x el coste de la capa de atención sin ganancia.
Padding	Dinámico por lote	<code>DataCollatorWithPadding</code> reduce el cómputo real sin afectar el resultado.
Variante de mBERT	cased	La <i>uncased</i> aplica <code>strip_accents=True</code> y destruye las tildes dentro del propio tokenizador.
Truncamiento	Activado	El poco texto que excede 128 tokens es cola de la distribución.

Punto crítico de configuración. Sería posible tomar todas las decisiones correctas de preprocesamiento y perderlas dentro del tokenizador sin ningún aviso. Usar `bert-base-multilingual-cased`, nunca la variante *uncased*.

7.1 Diferencias que condicionan la comparación

	mBERT	XLM-R
Vocabulario	~119k WordPiece	~250k SentencePiece
Preentrenamiento	Wikipedia, 104 idiomas	CommonCrawl (CC-100), ~2 TB
Registro coloquial	Escaso: Wikipedia es formal	Abundante: la web sí lo tiene
Cobertura de emojis	Casi nula	Parcial
Tokens por texto en español	Más (vocabulario menor)	Menos

Hipótesis del proyecto: XLM-R debería superar a mBERT en este corpus, porque el registro que lo caracteriza —jerga, coloquialismos, deformación ortográfica deliberada— está mucho mejor representado en CommonCrawl que en Wikipedia. El estudio de ablación debe confirmarlo o refutarlo; si lo refuta, eso también es un resultado.

8. Armonización con los datasets públicos

La primera etapa del *transfer learning* usa datasets públicos en español. El problema es que **ninguno comparte el esquema de cuatro clases del proyecto**.

Dataset	Tamaño	Esquema original	Mapeo propuesto
HatEval (SemEval-2019 T5)	~6,600	$HS \in \{0,1\} + TR + AG$	$HS=0 \rightarrow \text{No Tóxico} \cdot HS=1 \rightarrow \text{Discurso de Odio} \cdot HS=1 \text{ y } AG=1 \rightarrow \text{Amenazas}$
Spanish Hate Speech Superset	~13,000	binaria hate/no-hate	$0 \rightarrow \text{No Tóxico} \cdot 1 \rightarrow \text{Discurso de Odio}$
DETOXIS / IberBench	~3,460	$toxicity \in \{0,1,2,3\}$	$0 \rightarrow \text{No Tóxico} \cdot 1 \rightarrow \text{Lenguaje Ofensivo} \cdot 2-3 \rightarrow \text{Discurso de Odio}$
Multilingual Toxicity (CLANDESTINO)	variable	$toxic \in \{0,1\}$	$0 \rightarrow \text{No Tóxico} \cdot 1 \rightarrow \text{Lenguaje Ofensivo}$

8.1 Advertencias sobre el mapeo

- **El mapeo pierde información y la inventa.** Un dataset binario no distingue *Lenguaje Ofensivo* de *Discurso de Odio*; forzarlo a una de las dos introduce ruido. Vale la pena considerar entrenar la primera etapa con un esquema **binario o de tres clases** y reservar las cuatro para la segunda etapa sobre el corpus salvadoreño.
- **Sesgo dialectal.** HatEval y DETOXIS son de España y México. La segunda etapa (*domain adaptation*) es la que corrige eso, y el *ablation study* «antes vs después» es la evidencia que lo demuestra.
- **No mezclar en un solo lote.** El enfoque es secuencial: primero los datos públicos, después el corpus salvadoreño. Mezclarlos diluiría la adaptación al dominio local, que es el aporte diferencial del trabajo.
- **Aplicar la misma normalización** a los datos externos. Si no, el modelo vería dos distribuciones de texto distintas. El notebook incluye `armonizar_externo()` para eso.

9. Splits y configuración de entrenamiento

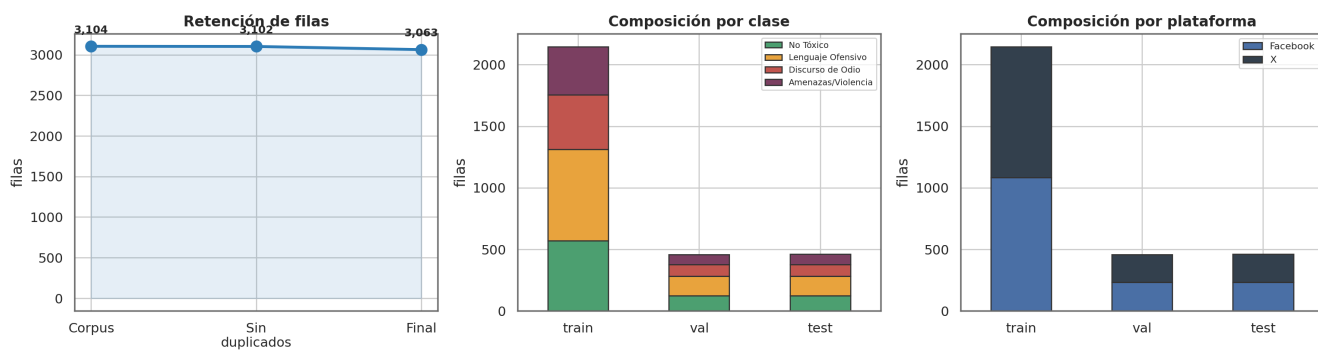


Figura 6. Retención de filas, composición por clase y composición por plataforma de cada conjunto.

Conjunto	n	No Tóxico	L. Ofensivo	D. de Odio	Amenazas
train	2,144	570	742	443	389
val	459	122	159	94	84
test	460	123	159	95	83
Total	3,063	815	1,060	632	556

Estratificación **doble, por clase y por plataforma** (8 estratos), semilla fija 42. El balance de plataforma queda en 50.6 / 49.4 en *train*, 50.5 / 49.5 en *val* y 50.4 / 49.6 en *test*. Se verificó ausencia total de solapamiento de identificadores y de textos entre conjuntos.

9.1 Configuración recomendada

Parámetro	Valor
Modelos	bert-base-multilingual-cased · xlm-roberta-base
Campo de texto	texto_modelo
num_labels	4
max_length	128, con padding dinámico
batch_size	16
learning_rate	2e-5 (rango de búsqueda 1e-5 a 5e-5)
Épocas	3 a 5, con <i>early stopping</i> sobre F1-macro en <i>val</i>
warmup_ratio · weight_decay	0.1 · 0.01
Función de pérdida	CrossEntropyLoss(weight=pesos_clase)
Métrica de selección	F1-macro sobre <i>val</i>

Clase	Peso para la pérdida
Amenazas/Violencia	1.3779
Discurso de Odio	1.2099
No Tóxico	0.9404
Lenguaje Ofensivo	0.7224

10. Resumen de decisiones

Aspecto	Decisión	Motivo
Tildes, mayúsculas, puntuación	Se conservan	Los modelos <i>cased</i> las modelan; son distintivas en español
Ortografía	No se corrige	Es señal, y evita <i>train/serve skew</i>
<i>Stopwords</i> , lematización	No se aplican	Recetas de bolsa de palabras; destruyen la sintaxis que usa la atención
Emojis	Demojizados a español	El vocabulario de mBERT casi no los cubre
@menciones y URLs	Token constante	Anonimiza preservando la señal estructural
Duplicados	Eliminados antes del <i>split</i>	Evitan métricas infladas
<code>max_length</code>	128	Cubre sobre el p99; la atención escala $O(n^2)$
Estratificación	Clase + plataforma	Las dos plataformas son distribuciones distintas
Desbalance	<code>class_weight</code> en la pérdida	Sin coste de datos; calculado solo sobre <i>train</i>

11. Siguiendo pasos

#	Tarea	Prioridad
1	Descargar y armonizar los datasets públicos en <code>data/external/</code> con <code>armonizar_externo()</code>	Alta
2	Ejecutar las celdas de tokenización con acceso a Hugging Face y confirmar <code>max_length</code>	Alta
3	<i>Fine-tuning</i> en dos etapas: datos públicos, luego corpus local. Comparar contra el baseline clásico del Anexo A del notebook	Alta
4	Reportar F1-macro global y desagregado por plataforma	Alta
5	Ablación <code>texto_modelo</code> vs <code>texto_agresivo</code> , en ambos modelos	Media
6	Generalización cruzada: entrenar en X, evaluar en Facebook, y viceversa	Media
7	<i>Fairness</i> : medir si la jerga local dispara falsos positivos	Alta
8	Recolección dirigida: 128 de Discurso de Odio, 214 de Amenazas/Violencia	Media

#	Tarea	Prioridad
9	Corregir en la tesina el supuesto sobre longitud X vs Facebook	Media

Estado del dataset

El dataset queda en **3,063 publicaciones**, estratificado por clase y plataforma, sin fugas entre conjuntos y con tres variantes de texto para permitir la ablación. Todo lo necesario para arrancar el entrenamiento está en `data/processed/`.

Nota de alcance: este informe cubre únicamente el análisis exploratorio y la preparación de los datos. Precisión, *recall* y F1 son métricas de predicción y requieren un modelo entrenado, de modo que pertenecen a la fase de **Modelado** de CRISP-DM y se reportarán en su propio documento. Las únicas cifras de F1 que aparecen aquí son los pisos triviales de la sección 3.1, que se calculan de la pura distribución de clases sin entrenar nada. El notebook incluye además un baseline clásico como **Anexo A**, fuera del alcance de este informe.